OOP Concepts

1. Encapsulation (التغليف)

عملية التغليف للتقليل من المخاطر عبر الـ Access modifier أي إعطاء الصلاحيات لتحديد إمكانية الوصول و التعديل.

Encapsulation keeps an object's data and the methods that work with that data together. It controls how other code can read or change the object's internal state.

Access control in Dart

- **Public:** Names are public by default and can be accessed outside their library.
- **Private:** A name beginning with `_` is private to its library, not only to its class.
- **Protected:** Dart has no `protected` keyword.

الـ Getter يسمح لنا بقراءة قيمة متغير بطريقة منظمة.

الـ Setter يسمح لنا بتعديل قيمة Property مع إمكانية وضع شروط قبل التعديل.

Example :

```
} class BankAccount
BankAccount(this.owner, {double startingBalance = 0})
;balance = startingBalance_ :

;final String owner
;double _balance

;double get balance => _balance

} set balance(double value)
} if (value < 0)
;throw ArgumentError('Balance cannot be negative')
{
;balance = value_
```

```
{

} void deposit(double amount)
} if (amount <= 0)
;throw ArgumentError('Deposit must be positive')
{
;balance += amount_
{

} void withdraw(double amount)
} if (amount <= 0)
;throw ArgumentError('Withdrawal must be positive')
{
} if (amount > _balance)
;throw StateError('Insufficient funds')
{
;balance -= amount_
{
{

} ()void main
final account = BankAccount('Mona', startingBalance: 10
;0)
;account.deposit(50)
print(account.balance); // 150
{
```

الـ Constructor هو دالة خاصة يتم استدعاؤها عند إنشاء Object من الـ Class.

```
BankAccount(this.owner, {double startingBalance = 0})
;balance = startingBalance_ :
```

التغليف يسمح لنا بالتحكم في البيانات وعدم السماح بتعديلها بطريقة عشوائية، من خلال Getters وSetters والدوال التي تحدد قواعد التعامل معها.

2. Inheritance (الوراثة)

إنشاء Class جديد يرث الخصائص والدوال من Class موجود مسبقًا.

Inheritance lets a new class (the child or subclass) reuse and specialize behavior from an existing class (the parent or superclass). is-a

The child can add methods or override inherited ones.

Example:

```
} class Animal
;Animal(this.name)

;final String name

;'String speak() => 'Some sound
{

} class Dog extends Animal
;Dog(String name) : super(name)

override@
;'!String speak() => 'Woof
{

} ()void main
;final dog = Dog('Rex')
print(dog.name); // Rex — inherited from Animal
print(dog.speak()); // Woof! — specialized by Dog
{
```

`Dog` inherits the initializer and `name` behavior from `Animal`, then provides its own version of `speak()`.

3. Composition (التركيب)

بناء Class باستخدام Objects من Classes أخرى بدل الاعتماد على الوراثة "has-a relationship".

Example

```
} class Engine
;'String start() => 'Engine started
{

} class Car
;Car(this.engine)

;final Engine engine

;()String start() => engine.start
{

} ()void main
;final car = Car(Engine())
print(car.start()); // Engine started
{
```

`Car` uses an `Engine`; a car is not a kind of engine. The engine can also be replaced with another compatible object.

4. Abstraction (التجريد)

إظهار ما يجب على الكلاس فعله، وإخفاء تفاصيل كيفية تنفيذه عن المستخدم. (hiding the implementation details).

Users of the object focus on **what it can do**, not every step of **how it does it**.

Example

```
} abstract class Vehicle
;()String start
{

} class ElectricCar extends Vehicle
override@
;'String start() => 'Battery system started
{

} ()void main
```

```
;()final car = ElectricCar
print(car.start()); // Battery system started
{
```

`Vehicle` specifies that every concrete vehicle must provide `start()`. `ElectricCar` supplies the implementation. The abstract class itself cannot be instantiated until its abstract methods are implemented.

5. Polymorphism (تعدد الأشكال)

التعامل مع أنواع مختلفة من Objects من خلال واجهة أو نوع مشترك، مع تنفيذ مختلف لكل نوع.

Different objects can respond to the same operation in their own way. Code can use a shared method without needing to know each object's exact class.

Example

```
;import 'dart:math' as math

} abstract class Shape
;()double area///يعني حددنا اسم الدالة ونوع القيمة التي س
 .ترجعها، لكن لم نحدد طريقة حساب المساحة
{

} class Circle extends Shape
;Circle(this.radius)

;final double radius

override@
;double area() => math.pi * radius * radius
{

} class Rectangle extends Shape
;Rectangle(this.width, this.height)

;final double width
;final double height
```

```
override@
;double area() => width * height
{

} void printArea(Shape shape)
;print(shape.area())
{

} ()void main
printArea(Circle(2)); // About 12.57
printArea(Rectangle(3, 4)); // 12
{
```